

Brzozowski 微分でつくる正規表現エンジン

– たった2つの関数・約30行が実用エンジンになる –

Nextbeat Osaka Lab #1 「Claude Codeで作る、正規表現エンジン」理論編
株式会社ネクストビート

このスライドは単独で読めます。手を動かすには [README.md](#) のセットアップへ。

このスライドの位置づけ

- 本編は3段構成: ①ReDoS を見る（掴み）→ ②**設計を固める（このスライド）** → ③設計表を AI に投入して実装（ハンズオン本体）
- 人間が握るのは**基本設計**。コードは AI（Claude Code / Codex）が書く
- 設計表（[SPEC.md](#)）とコードが1対1だから、それをそのままプロンプトに貼るだけで実装が立ち上がる
- このあと約15分、その「設計表」の中身をここで合意する

事件: 標準の RegExp が長い入力で固まる

```
/^(a+)+$/ .test("a".repeat(n) + "X")
```

- n を1つ増やすたびに時間がおよそ倍になる（実測値は当日の実演・README 参照）
- これが **ReDoS**（Regular expression Denial of Service）
- 対して本エンジン（自作）は同じ入力をミリ秒未満で返す
- `pnpm bench:redos <n>` で再現可能（`n` は 30 を超えると `--force` が必要）

なぜ固まるのか① – バックトラックの直観

標準 RegExp は「試して、ダメなら戻ってやり直す」＝バックトラッキング型

`(a+)+` に `"aaaa"` を食べせると、エンジンは「a の列を何グループに分けるか」を試す:

```
(aaaa) / (aaa)(a) / (aa)(aa) / (aa)(a)(a)
(a)(aaa) / (a)(aa)(a) / (a)(a)(aa) / (a)(a)(a)(a)
```

末尾が `x` だと全部の分け方が失敗するまで戻り続ける。
マッチする時は速い。マッチしない時に全経路を確認しに行く。

なぜ固まるのか② – 分け方は $2^{(n-1)}$ 通り

長さ n の **a** の列を連続グループに分割する方法 = $2^{(n-1)}$ 通り（各文字間で「切る/切らない」）

n	分け方
4	8
10	512
28	約1.3億
40	約5500億

入力が1文字増えるたび**2倍**。これが指数爆発の正体。

処方箋: 「戻らない」 エンジン = DFA

解決策は昔から知られている: **DFA** (決定性有限オートマトン、deterministic finite automaton)

1文字読むごとに次の状態が一意に決まり、戻る操作を持たない。入力長 n に対して $O(n)$ 。

でも教科書の作り方は重い: 正規表現 $\rightarrow$ NFA (Thompson構成) $\rightarrow$ 部分集合構成 $\rightarrow$ DFA 最小化...

今日の主役: Brzozowski 微分 (1964) なら、この全部が**再帰関数2つ・約30行**に化ける。
オートマトンの予習は不要。

発想の転換: 正規表現そのものが「状態」

問い: `abc` にマッチ中、`a` を読み終えた状態って何？

答え: 「残りは `bc` にマッチすればいい」という状態

```
abc --a--> bc --b--> c --c--> ε(空文字OK) ✓
```

マッチの途中経過は「残りの正規表現」で表せる。状態＝正規表現そのもの。

この「1文字食って残りの正規表現を返す」操作を微分 (derivative) と呼ぶ。

記号の読み方（この2つだけ）

記号	読み	意味	コードでは
$v(r)$	ニュー	r は空文字列 ϵ を受理するか	<code>nullable(r): boolean</code>
$\partial c(r)$	デル・文字 c での微分	r を文字 c で微分した残りの正規表現	<code>derivative(r, c): Re</code>

ついでに: $\emptyset$ （空集合＝何も受理しない）、 ϵ （イプシロン＝空文字列だけ受理）、 $\wedge$ ＝かつ、 $\vee$ ＝または

数学の微分との関係は気にしなくていい。**ただの関数名。**

「微分」という名前の理由

積の微分則 $(fg)' = f'g + fg'$ は「左を微分した項」 + 「右を微分した項」の和。

後で出てくる接続則 $\partial c(rs) = \partial c(r) \cdot s \mid (v(r) ? \partial c(s) : \emptyset)$ も、
第1項が「左を微分した項」、第2項（ $v(r)$ が有効化する）が「右を微分した項」——構造は同型。

名前は飾りでなく、規則表がこの形で書ける理由そのもの。

部品: 正規表現の最小コアは6種類だけ

```
Re = ∅ (Empty)           -- 何も受理しない
    | ε (Eps)             -- 空文字列だけ受理
    | Class(set, neg)     -- 1文字。'a' も [a-z] も . も全部これ
    | Concat(left, right) -- 接続 rs
    | Alt(left, right)    -- 選択 r|s
    | Star(body)          -- 星 r*
```

`+ ? {n,m} ( )` は全部このコアへ脱糖: `r+ → r·r*`、`r? → r|ε`。

TypeScript では判別タグ付きユニオン (`src/ast.ts`)。中核は**switch 6分岐**で閉じる。

$v(r)$ — 空文字列を受理するか

r	$v(r)$	理由
$\emptyset$	false	何も受理しない
ε	true	空文字列そのもの
Class	false	必ず1文字消費する
r^*	true	0回マッチ ($=\varepsilon$) を含む
rs	$v(r) \wedge v(s)$	両方が空を許して初めて空
$r s$	$v(r) \vee v(s)$	どちらかが空を許せば空

この表が `src/derivative.ts` の `nullable` の switch **6分岐と1対1**。

$\partial c(r)$ – 文字 c で微分する

「 r がマッチする文字列のうち、先頭が c のものから c を1文字剥がした残り」にマッチする正規表現。

r	$\partial c(r)$
$\emptyset$	$\emptyset$
ε	$\emptyset$
Class	$c \in \text{class} ? \varepsilon : \emptyset$
r^*	$\partial c(r) \cdot r^*$
rs	$\partial c(r) \cdot s \mid (\mathbf{v(r) ? \partial c(s) : \emptyset}) \star$
$r s$	$\partial c(r) \mid \partial c(s)$

6行中5行は「見たまま」。★の接続則だけが非自明 → 次のスライドで。

唯一の非自明ポイント: 接続則の★

$\partial c(rs) = \partial c(r) \cdot s \mid (v(r) ? \partial c(s) : \emptyset)$ を日本語に分解:

rs の先頭の c は、どっちが食べた？

- └─ 左の r が食べた → 残りは $\partial c(r) \cdot s$ (常にある枝)
- └─ 左の r を「0文字でスキップ」して右の s が食べた
→ 残りは $\partial c(s)$
ただし r が空文字列を許す ($v(r)=\text{true}$) 時だけ！

この v 項を忘れても大半のテストは通ってしまう（壊れるのは「左が ε を含む接続」だけ）。

手トレース: $a?b$ を $'b'$ で微分する

$a?b$ は脱糖して $(a|\epsilon) \cdot b$ 。入力 $"b"$ を1文字目 b で微分する:

```

 $\partial b( (a|\epsilon) \cdot b )$ 
=  $\partial b(a|\epsilon) \cdot b \mid (v(a|\epsilon) ? \partial b(b) : \emptyset)$       ← 接続則
 $\partial b(a|\epsilon) = \partial b(a) \mid \partial b(\epsilon) = \emptyset \mid \emptyset = \emptyset$       ← 第1項は消える
 $v(a|\epsilon) = \text{false} \vee \text{true} = \text{true}$       ← ★第2項が生きる
=  $\emptyset \cdot b \mid \partial b(b) = \emptyset \mid \epsilon = \epsilon$ 
 $v(\epsilon) = \text{true} \rightarrow "b" \text{ は受理される } \checkmark$ 

```

v 項を忘れていたら結果は $\emptyset$ のまま、 $"b"$ は不受理になっていた。

このスライドがクライマックス

- `"ab"` は v 項なしでも通る ($\partial a(a?b)=b$ 、 $\partial b(b)=\varepsilon$)
- でも `"b"` (a を0回スキップ) は **v 項だけが結果を生む唯一の経路**
- だから他のテストが緑のまま、この1件だけ静かに壊れる
- `tests/spec.test.ts` の「 $a?b$ トラップ」がこれを検出する

AI の生成コードが `a?b` テストで赤になったら、疑う場所は連接則の v 項一択。

マッチング＝微分の畳み込み

エンジン全体はこの1行:

```
function match(re: Re, input: string): boolean {  
    return nullable(  
        [...input].reduce<Re>((state, ch) => derivative(state, ch.codePointAt(0)!), re),  
    );  
}
```

1文字ずつ微分して、最後に「残りが空文字列を受理するか」を見るだけ。

「戻る」操作がコード上どこにも存在しない → バックトラック不能 → この時点で既に線形。

メモ化したら、勝手に DFA になっていた

`match` は毎回微分を計算し直す。→ 微分結果をキャッシュしたら？

`ab*` の全微分を書き出すと:

$$\begin{array}{ll} \partial a(ab^*) = b^* & \partial b(ab^*) = \emptyset \\ \partial a(b^*) = \emptyset & \partial b(b^*) = b^* \quad \leftarrow \text{自分に戻る!} \\ \partial a(\emptyset) = \emptyset & \partial b(\emptyset) = \emptyset \quad \leftarrow \emptyset \text{は何を食べても}\emptyset \end{array}$$

出てくる正規表現は $\{ab, b, \emptyset\}$ の3つで打ち止め**。

状態 = 正規表現、遷移 = 微分、受理 = v 。これは DFA の定義そのものの。

遅延DFA: コードにすると数十行

`src/match.ts` の `LazyDfa` の骨格（抜粋）:

```
step(id, c) {  
  const cached = row.get(c);  
  if (cached !== undefined) return cached;           // 2回目以降: 表引きだけ  
  const next = this.intern(derivative(this.states[id], c)); // 初回だけ微分  
  row.set(c, next);  
  return next;  
}
```

「遅延」＝全遷移を先に作らず、実際に来た文字の分だけオンデマンド構築。
実行時は Map 引きのみ。状態＝「ここまで読んだ時点に残っている正規表現」という直観そのものの。

落とし穴: このままだと状態が爆発する

微分は新しい正規表現を**生み続ける**。素朴にノードを作ると:

$\varepsilon \cdot b^*$ と b^*	← 意味は同じ、構造が違う
$r r$ と r	← 意味は同じ、構造が違う
$a b$ と $b a$	← 意味は同じ、構造が違う

キャッシュのキーが一致しない → メモが永遠にヒットしない → **状態が無限増殖**。
 $(a^+)^+$ がまさにこれで爆発する (ReDoS を殺すはずのエンジンが別の形で死ぬ)。

もう1つの罠: Map のキーにオブジェクト参照を使うと、同じ構造でも別状態になる。
必ず「正規化後の構造文字列」(**canonicalKey**) をキーにする。

処方箋: ACI 正規化（スマートコンストラクタ）

生のオブジェクトリテラルでノードを作らず、必ず `mk*` 経由で生成時に等価な形へ潰す:

関数	規則
mkAlt	<code>∅ r = r</code> （吸収） / <code>r r = r</code> （冪等I） / <code>r s = s r</code> （可換C→キーでソート） / フラット化（結合A）
mkConcat	<code>∅ · r = r · ∅ = ∅</code> （零元） / <code>ε · r = r · ε = r</code> （単位元） / 右結合に固定
mkStar	<code>∅* = ε</code> / <code>ε* = ε</code> / <code>(r*)* = r*</code>

ACI = Associativity · Commutativity · Idempotence。この最小限の正規化で状態が有限個に収まる。

実測: 入力10倍 → 時間10倍、状態数は一定

`pnpm bench:linear` の出力例（数値は当日実測に差し替え）:

`pattern = (a+)+` ← 標準RegExpが長い入力で固まるやつ

入力長	所要(ms)	DFA状態数
10,000	...	2 (一定)
100,000	...	2 (一定)
1,000,000	...	2 (一定)
10,000,000	約200	2 (一定)

読み方: ①入力10倍→時間ほぼ10倍 = **$O(n)$** ②状態数が入力長に依存しない = 正規化が効いている証拠。

あえて入れないもの: 後方参照 `\1`

`(.+) \1` (同じ文字列の2回繰り返し) のような後方参照は、今日のエンジンには**入れない**。

- 後方参照は正則言語の表現力を**超える** → 有限の状態では原理的に表現できない → DFA化が不可能
- 標準 RegExp がバックトラッキング型なのは、まさに `\1` などをサポートするため **表現力と引き換えに ReDoS を抱え込んだ**
- 逆に言うと: **入れないからこそ線形時間と ReDoS 耐性が「頑張る」ではなく「保証」になる**
- 実例: Google RE2 / Rust regex / Go regexp も同じ判断で `\1` 非対応

「何を入れないかで品質が決まる」——本日の設計判断の核心。

まとめ & このあとやること

1. 状態 = 正規表現そのもの。遷移 = 微分 ∂ 。受理 = v 。メモ化したら **DFA**
2. 非自明なのは接続則の v 項ただ1つ (`a?b` トラップで検出可能)
3. 正規化 (ACI) が状態を有限に抑え、後方参照を入れない判断が線形を**保証**する

このあと: [SPEC.md](#) の規則表を [AGENT PROMPTS.md](#) ①のプロンプトで
AI に投入 → `pnpm test` の赤が緑になるのを見届ける → ②正規化 → 実食 → クローリング。

参考文献: J.A. Brzozowski, "Derivatives of Regular Expressions" (JACM 1964) /
Owens, Reppy, Turon, "Regular-expression derivatives re-examined" (JFP 2009) / 本リポジ
トリ `SPEC.md`